

Paradigmas de Programación + Arquitectura Cliente-Servidor

Los paradigmas son enfoques teóricos y estilos fundamentales para el desarrollo de software. Estos se dividen en dos vertientes principales, determinadas por el grado de interacción directa que requieren con las abstracciones de bajo nivel del computador.

Paradigmas Imperativos

```
#include <stdio.h>

int main()
{
    int a, b;

    printf( "Introduzca primer numero (entero): " );
    scanf( "%d", &a );
    printf( "Introduzca segundo numero (entero): " );
    scanf( "%d", &b );

    if ( a + b > 0 )
        printf( "LA SUMA SI ES MAYOR QUE CERO." );
    else
        printf( "LA SUMA NO ES MAYOR QUE CERO." );

    return 0;
}
```

Nota al pie 1: Representación conceptual de un flujo de control imperativo, donde cada instrucción dicta un paso secuencial hacia el resultado.

El enfoque imperativo implica un desglose exhaustivo y secuencial de los pasos exactos que necesita realizar el computador para producir un resultado. Se caracteriza por el uso intensivo de la asignación y mutación de variables, estructuras de control de flujo (bucles y condicionales), y en lenguajes de más bajo nivel, la manipulación directa de direcciones en memoria.

Ejemplos de paradigmas imperativos:

1. **Programación Procedimental:** Este modelo se fundamenta en la

```
int total = 0;
for(int i = 0; i < 10; i++){
    total++;
    printf("%i\n", total);
}

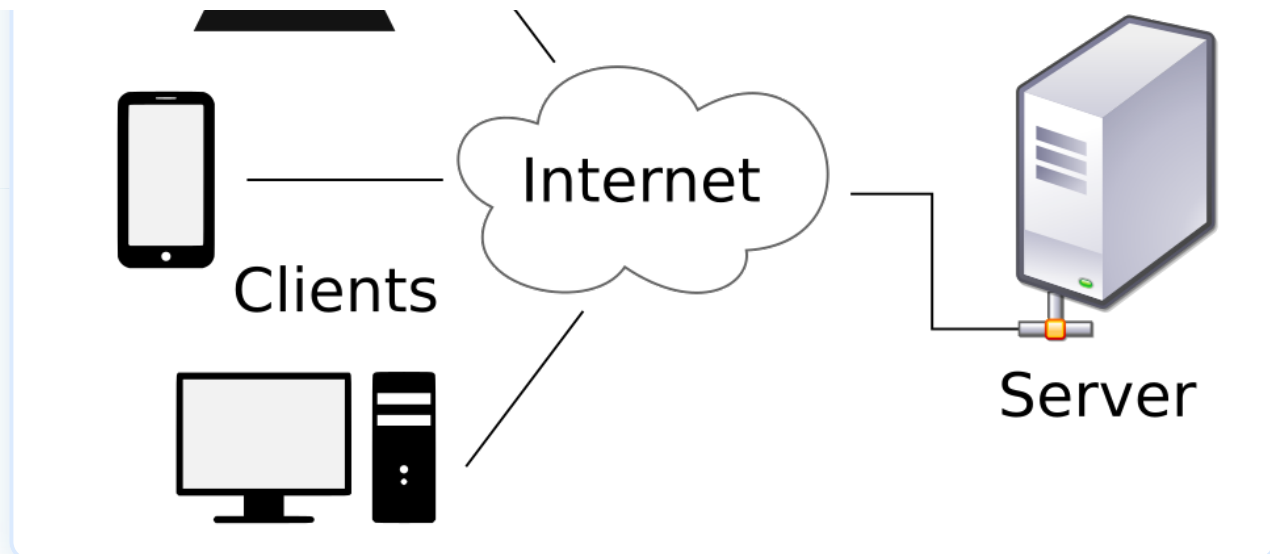
> 0
> 1
> 2
> 3
> 4
> 5
> 6
> 7
> 8
> 9
```

Nota al pie 2: La declaratividad abstrae el "cómo". El desarrollador se enfoca en el estado final deseado, delegando la ejecución algorítmica al motor del lenguaje.

Cuando el manejo de bajo nivel pierde relevancia frente a la necesidad de ensamblar sistemas complejos, la declaratividad se vuelve el enfoque ideal para trabajar sobre niveles más altos de abstracción. Consiste en describir **qué** es lo que se quiere conseguir en lugar de detallar **cómo** lograrlo. En este estilo, el desarrollador describe el resultado deseado de forma consistente con las reglas del lenguaje, confiando en que el motor subyacente infiera y ejecute los pasos intermedios apropiados.

Ejemplos específicos de paradigmas declarativos:

1. **Programación Lógica:** El programa se expresa como un conjunto de hechos y reglas lógicas. El motor deductivo del lenguaje se encarga de encontrar las respuestas a las consultas realizadas.
2. **Programación Funcional:** Trata la computación como la evaluación de funciones matemáticas, evitando cambiar el estado y mutar los datos, lo que elimina efectos secundarios indeseados.
3. **Programación Reactiva:** Se centra en los flujos de datos



Nota al pie 3: Arquitectura de red estandarizada que ilustra la solicitud de recursos desde múltiples clientes hacia un servidor centralizado.

Esta arquitectura es indispensable cuando una aplicación necesita manejar una base de datos centralizada (para evitar inconsistencias) o realizar procesamiento intensivos que sobrepasarían las capacidades del dispositivo cliente. Es el modelo estándar para aplicaciones bancarias, videojuegos multijugador o redes sociales.

En términos materiales, el cliente y el servidor suelen ser procesos que se ejecutan en computadores distintos conectados a través de una red (como Internet o una intranet). Esto hace que la conectividad sea un requisito obligatorio y un posible punto de fallo. Además, el servidor debe estar diseñado para manejar peticiones concurrentes de múltiples clientes y ser altamente tolerante a fallos.

Arquitectura de tres capas

La arquitectura de tres capas es la evolución natural del modelo cliente-servidor tradicional. Surge para resolver problemas de escalabilidad y mantenimiento, aislando las distintas responsabilidades del software en tres niveles lógicos (y frecuentemente físicos) independientes:

Creación y configuración de un sitio web

La implementación y despliegue de un ecosistema web contemporáneo se articula a través de diversas fases técnicas y operativas:

1. **Planificación y Diseño:** Comprende la definición de la arquitectura de información, el diseño de la experiencia de usuario (UX) y la creación de interfaces visuales (UI) para garantizar la coherencia funcional del proyecto.
2. **Desarrollo y Programación:** Representa la etapa de construcción técnica del software, estructurada en dos dimensiones fundamentales:
 - *Frontend:* Codificación de la capa de presentación mediante lenguajes declarativos e imperativos (HTML, CSS y React), permitiendo la interacción fluida del usuario final.
 - *Backend:* Creación de la lógica de negocio centralizada y gestión de la persistencia de datos mediante motores de bases de datos robustos.
3. **Adquisición de Dominio y Alojamiento (Hosting):** Fase crítica para la gestión de activos digitales y la puesta en línea del sistema:
 - *Dominio:* Registro de un identificador alfanumérico legible (ej. *tusitio.com*) que los servidores DNS traducen unívocamente a una dirección IP de red.
 - *Hosting:* Configuración de la infraestructura computacional, ya sea física o virtual, encargada de mantener la disponibilidad del sitio de forma ininterrumpida.
4. **Configuración del Servidor y Despliegue:**
 - Transferencia de los activos digitales desde entornos de desarrollo hacia el servidor de producción mediante protocolos de integración continua o transferencia segura